Alpha Fitness Operación y despliegue

Edición: 2.0 · 19 de octubre de 2026.

Revisión: base `06d31c2984133a9596bff0db21935399eb8c658c` más árbol local del 19 de octubre, identificado en REVISION.json.

Dirigido a: responsable técnico de instalación, actualización, respaldo y continuidad.

Los procedimientos siguientes son una guía para el entorno destino. No acreditan despliegue ni restauración productivos ya ejecutados. El Manual técnico describe arquitectura/reglas; el Diccionario describe datos. No aplicar comandos de reinicialización a una instalación con información que se deba conservar.

1. Preparación

Definir antes de instalar: revisión de código, ubicación, dominio, responsable de infraestructura, motor/base, servidor web y PHP, correo, almacenamiento de respaldos y quién atenderá incidentes. Utilizar una base dedicada y separar desarrollo, prueba y producción; un worktree no aísla bases, puertos ni cachés.

Comprobación	Método	Resultado esperado
Código y cambios locales	`git status --short --branch` y `git rev-parse HEAD`	Revisión identificada; cambios locales preservados
PHP y extensiones	`php -v`, `php -m`	PHP compatible, PDO del motor y GD/WebP
Dependencias PHP	`composer check-platform-reqs` tras instalar	Sin requisitos ausentes
Toolchain frontend	`node --version`, `npm --version`	Compatible con locks/Vite
Conexión y esquema	`php artisan migrate:status`	Base correcta y migraciones identificadas
Recursos	`public/build/manifest.json` y navegador	Archivos presentes y servidos por HTTP

Ejecutar comandos desde la raíz del proyecto. En el Windows verificado, PHP está en `C:/xampp/php/php.exe`; PowerShell permite `& 'C:/xampp/php/php.exe' artisan ...`. Si npm activa una restricción de scripts, utilizar `npm.cmd`. Composer también debe resolver el PHP previsto; no mezclar CLI y servidor con versiones/extensiones distintas.

2. Instalación nueva

Este procedimiento exige una copia nueva y base vacía. Mantener `composer.lock` y `package-lock.json`; instalar versiones fijadas, sin `update` ni omisión de requisitos.

1. Obtener el código de la revisión elegida en un directorio dedicado.
2. Ejecutar `composer install` y `npm ci`. En producción se puede preparar un artefacto con `composer install --no-dev --optimize-autoloader` y assets compilados desde un entorno de construcción con dependencias de desarrollo.
3. Copiar `.env.example` a `.env` solo cuando no exista configuración. En PowerShell:

```
if (Test-Path -LiteralPath '.env') {
    throw 'Ya existe .env; conservarlo y configurar la instalación existente.'
}
Copy-Item -LiteralPath '.env.example' -Destination '.env'
```

4. Completar URL, zona horaria, base y servicios. Para SQLite, crear el archivo únicamente si no existe y configurar `DB_CONNECTION=sqlite`; `DB_DATABASE` puede indicar ruta absoluta:

```
if (-not (Test-Path -LiteralPath 'database/database.sqlite')) {
    New-Item -ItemType File -Path 'database/database.sqlite' | Out-Null
}
```

Para MySQL u otro motor, crear base/usuario dedicados, instalar driver PDO y configurar su conexión. La presencia de configuración no acredita compatibilidad de producción probada en esta entrega.

5. Generar clave **solo en instalación nueva**: `php artisan key:generate`.
6. Revisar los seeders y ejecutar `php artisan migrate --seed`. Crean catálogos; la migración de hardening incorpora planes iniciales. No crean una cuenta administradora de contraseña pública.
7. Ejecutar `php artisan admin:create` y responder al prompt oculto de contraseña. No pasar secretos por `--password` ni guardarlos en documentación. El comando impide un segundo administrador.
8. Ejecutar `npm run build`. Para desarrollo local, iniciar `php artisan serve --host=127.0.0.1 --port=8000` y abrir la URL indicada; Vite `npm run dev` es opcional para trabajo frontend.
9. Completar comprobaciones del apartado 6 y configurar correo antes de depender de invitaciones/recuperación.

`composer setup` combina instalación, generación de clave, migración con seeders y build. Su alcance corresponde a una instalación nueva; no usarlo para actualizar una base existente. `composer dev` inicia servidor, colas, logs y Vite, y requiere un entorno de desarrollo aislado. El servidor de Artisan no es la publicación productiva.

3. Configuración

Los valores de ejemplo no son credenciales ni decisiones institucionales. Tras cambiar configuración, regenerar su caché y comprobar la lectura efectiva sin imprimir secretos.

Grupo	Variables principales	Decisión
Aplicación	`APP_NAME`, `APP_ENV`, `APP_DEBUG`, `APP_URL`	Producción: environment production, debug false y URL HTTPS real
Clave	`APP_KEY`, `APP_PREVIOUS_KEYS`	Custodiar; no regenerar para actualizar ni reparar sesiones
Idioma/fechas	`APP_TIMEZONE`, `APP_LOCALE`, `APP_FALLBACK_LOCALE`	Ejemplo America/El_Salvador y es; no reinterpretar historia sin análisis
Base	`DB_CONNECTION`, `DB_DATABASE`, `DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD`, `DB_URL`	Base dedicada; DB_URL puede prevalecer sobre parámetros separados
SQLite	`DB_FOREIGN_KEYS`	Mantener integridad activada y ruta fuera del acceso web
Sesión	`SESSION_DRIVER`, `SESSION_LIFETIME`, `SESSION_ENCRYPT`, `SESSION_SECURE_COOKIE`, `SESSION_HTTP_ONLY`, `SESSION_SAME_SITE`, `SESSION_COOKIE`	Defaults database, 120 minutos, cifrada, HttpOnly y Lax; Secure en producción
Caché	`CACHE_STORE`, `CACHE_PREFIX`	Namespace/store coherente; afecta límites de solicitudes
Colas	`QUEUE_CONNECTION` y variables del driver	Database por defecto; no implica que todo correo se encole
Correo	`MAIL_MAILER`, `MAIL_SCHEME`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	log/array no entregan; configurar transporte/remitente reales
Google	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REDIRECT_URI`	Opcional; callback exacto `/cliente/google/callback`
Privacidad	`PRIVACY_RESPONSIBLE_NAME`, `PRIVACY_CONTACT_EMAIL`	Completar con datos institucionales antes de publicación
Registro	`LOG_CHANNEL`, `LOG_STACK`, `LOG_LEVEL`, `LOG_DAILY_DAYS`	Retención y acceso restringido; daily solo si se selecciona

Correo

Configurar proveedor autorizado y comprobar desde una cuenta de ensayo invitación, enlace de restablecimiento y comprobante. Validar recepción y enlace completo sin publicar tokens. El envío de comprobantes actual es síncrono: arrancar un worker no convierte `log` en correo real. Consultar fallos acotados y clase de excepción; no copiar credenciales ni mensajes completos al pedir soporte.

En modo `log`, el mensaje puede aparecer con enlace privado en logs; restringir su acceso. La interfaz avisa que no se entregó por correo. Reenviar desde el directorio de personal solo cuando la cuenta esté activa y pendiente; respetar límite de reintentos/caducidad. No crear contraseñas compartidas para eludir la invitación.

Google

Si se habilita, registrar el dominio y callback exactos del entorno en el proveedor, introducir credenciales privadas y probar consentimiento, cancelación, correo verificado, cuenta desactivada y regreso al sitio. Mantener alternativa de acceso por correo. Vincular una cuenta local existente desde configuración con contraseña; no vincular automáticamente por dirección coincidente.

Privacidad y modo de demostración

Los nombres/contactos institucionales no se pueden deducir del código. La revisión local comunica los pendientes al administrador; cerrar esos campos exige información válida del responsable. Las ventas de interfaz son simuladas y consumen stock: utilizar catálogo/base de ensayo para demostraciones y no contaminar el inventario operativo. Los pagos manuales de membresía sí entran en los ingresos registrados. No convertirlos en pruebas dentro de la base real.

4. Actualización con datos existentes

1. Identificar código, cambios locales, configuración, motor, rutas de archivos y migraciones aplicadas. Revisar los efectos de las pendientes antes de ejecutarlas.
2. Preparar revisión y build en ensayo con una copia protegida y servicios externos de prueba. Preservar `.env`, `APP_KEY`, datos y archivos cargados.
3. Obtener respaldo coherente y comprobarlo como indica el apartado 7. La vuelta al código anterior puede ser insuficiente si cambió esquema o hubo operaciones después del respaldo.
4. Acordar ventana, bloquear escrituras y detener ordenadamente consumidores propios que escriban. `php artisan down` limita solicitudes, pero por sí solo no detiene procesos CLI/colas ni conexiones externas.
5. Publicar el código y dependencias de la revisión preparada. No sobrescribir configuración del sitio ni retirar imágenes cargadas con un reemplazo ciego del directorio.
6. Ejecutar `php artisan migrate --force` solo sobre la base destino confirmada. No repetir seeders por rutina y no ejecutar `migrate:fresh`, `db:wipe` ni `migrate:refresh` con información existente.
7. Generar cachés de la revisión/configuración final mediante `php artisan config:cache` y `php artisan view:cache`; si se utiliza caché de rutas, comprobar `php artisan route:cache`. Reiniciar servicios persistentes/OPcache propios según servidor; workers existentes mediante su procedimiento

supervisado.

8. Comprobar salud, recursos, sesión, roles y lecturas. Abrir con `php artisan up` solo al resolver fallos materiales y validar las operaciones relevantes con datos de ensayo.

No rotar `APP_KEY` para actualizar. No resolver conflicto de migración borrando cuentas o historial. `migrate:rollback` puede eliminar datos y restaurar cascadas; no es un plan de recuperación general.

Cambios de esta edición

La migración `2026_10_19_100000_add_simulated_payments_to_ventas.php` agrega bandera indexada, efectivo recibido, cambio y referencia. Las ventas existentes mantienen `pago_simulado=false`. La aplicación debe usar el esquema actualizado antes de servir las nuevas consultas. La migración se aplicó y verificó en SQLite local después de respaldo; eso no acredita su ejecución en un alojamiento externo.

Para instalaciones mucho más antiguas, revisar también separación de tokens, restricción de borrado financiero y asignador por ID. Los tokens antiguos se invalidaron por diseño y una asignación histórica con nombre sin ID no se convierte automáticamente en un vínculo de acceso.

5. Publicación y archivos

El servidor web debe dirigir solicitudes a `public/index.php`, con document root en `public`. Proteger raíz, `.env`, bases, vendor, logs y respaldos; conceder escritura al proceso únicamente donde corresponda. Laravel necesita escritura en `storage` y `bootstrap/cache`. Alpha Fitness también carga imágenes en `public/images/avatars`, `productos` y `ejercicios`; conservarlas entre revisiones. Estas recomendaciones siguen la guía oficial Laravel 12.

Usar HTTPS real y comprobar su reconocimiento por Laravel cuando exista proxy. La cookie Secure no funciona sobre HTTP de producción. Configurar tamaños de carga y memoria compatibles con límites de aplicación e imágenes, sin ampliar por rutina. No reutilizar la configuración local con debug activo.

`public/build` debe contener manifest y archivos compilados correspondientes. No dejar `public/hot` apuntando a un Vite de desarrollo en una publicación. Los assets de código, insignias y catálogo se versionan; fotos cargadas por usuarios requieren persistencia y respaldo. `storage:link` no sustituye el respaldo de `public/images`.

6. Comprobaciones y aceptación

Área	Comprobación	Éxito observable
Arranque	`/up`, login y `/informacion`	Respuesta correcta y assets sin fallos
Roles	Acceso propio y URL directa ajena con cuentas de ensayo	Navegación correcta; 403 en funciones ajenas
Personal	Alta e invitación de prueba con transporte válido	Enlace recibido; contraseña establecida; acceso habilitado
Ventas	Efectivo/cambio, tarjeta rechazada, reintento aprobado, doble envío	Stock una sola vez; simulación visible; rechazo sin venta
Reportes	Rango con venta simulada y registro manual de ensayo	Simulación excluida de todos los importes; PDF/Excel legibles
Membresías	Solicitud/activación y segundo intento	Pago único y fechas correctas; solo Secretaria activa
Progreso	Registro propio, filtro y consulta de entrenador ajeno	Rango global estable; ajeno rechazado
Móvil	320-390 px, teclado, foco, menú y recibo	Sin controles tapados ni desbordamiento relevante
Privacidad	Configuración institucional y exposición pública	Responsable/contacto válidos; archivos privados inaccesibles

No ejecutar cobros ficticios en bases operativas. La prueba SMTP/OAuth depende de credenciales y servicios autorizados. La verificación de navegador usa datos sintéticos; no incorporar fotos o nombres de personas reales a capturas de documentación.

Para desarrollo/ensayo con dependencias instaladas:

```
php vendor/phpunit/phpunit/phpunit
node tools/frontend_regression_check.cjs
npm run build
```

`phpunit.xml` selecciona SQLite en memoria y transportes de prueba. Antes de ejecutar, comprobar que no existe caché de configuración productiva ni variables externas que cambien el destino. `composer test` limpia caché de configuración; reservarlo al contexto previsto.

La revisión móvil requiere primero generar la base sintética y su configuración privada; después ejecuta el navegador:

```
php tools/prepare_mobile_qa.php
node tools/mobile_review_check.cjs
```

El primer script prepara una SQLite temporal y `storage/app/private/mobile-qa.json`, que contiene credenciales ficticias privadas; no publicar ese archivo. El segundo la consume, verifica el destino y

levanta servidor de prueba en 127.0.0.1:8034. `ALPHA_QA_PHP` selecciona PHP (default `C:/xampp/php/php.exe`) y `ALPHA_QA_BROWSER` el ejecutable del navegador (default Chrome en `C:/Program Files/Google/Chrome/Application/chrome.exe`). Configurarlas si la instalación cambia y confirmar puerto libre y build actualizado. Revisar scripts/precondiciones antes de ejecutarlos en otra máquina. No se acredita una prueba si no ejecutó casos pertinentes.

7. Respaldo coherente

El responsable debe fijar frecuencia, retención, cifrado, destino, acceso, objetivo de pérdida admisible (RPO) y tiempo de recuperación (RTO). No hay valores institucionales aprobados ni servicio automático de respaldo implementado en esta entrega.

Respaldar conjuntamente base consistente, imágenes cargadas, configuración/APP_KEY bajo custodia separada y revisión de código/locks/manifest. Conservar fecha, motor, revisión, migraciones y hash de archivos. No guardar copias en `public`, Git, capturas o una carpeta de descargas compartida.

SQLite

No copiar solo el archivo principal mientras haya escritores: pueden existir datos en WAL o journals. Usar API de respaldo de SQLite o `VACUUM INTO` a un archivo nuevo; coordinar la ventana con las imágenes para coherencia del conjunto. Ver documentación de SQLite Backup API y VACUUM INTO.

Ejemplo PHP orientativo para un procedimiento controlado, con origen/destino definidos por el responsable; no es un comando ejecutado automáticamente por la aplicación:

```
<?php
$origen = realpath($argv[1] ?? '');
$destino = $argv[2] ?? '';
if (!$origen || !is_file($origen) || !$destino || file_exists($destino)) {
    throw new RuntimeException('Origen válido y destino nuevo requeridos.');
}
$db = new PDO('sqlite:'.$origen, null, null, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
]);
$db->exec('VACUUM INTO '.$db->quote($destino));
$copia = new PDO('sqlite:'.$destino);
if ($copia->query('PRAGMA quick_check')->fetchColumn() !== 'ok') {
    throw new RuntimeException('La copia no superó quick_check.');
}
```

Confirmar antes que la ruta destino resuelta esté en almacenamiento privado previsto y que no coincida con origen; los errores de ruta/espacio deben detener el procedimiento. `quick_check` es una comprobación de integridad, no una restauración ni reconciliación completa de negocio. Verificar también relaciones, conteos de control y un ensayo de recuperación. El respaldo local de esta edición usó PDO con `VACUUM INTO`; no dependió de la clase PHP SQLite3.

Otros motores

Utilizar herramienta y opciones de respaldo consistente del motor/versiones elegidos, protegiendo credenciales y verificando su restauración. No trasladar la receta de archivo SQLite a MySQL. La prueba y procedimiento exactos del motor de producción quedan a cargo del despliegue; no se ejecutaron en esta revisión.

8. Restauración y reversión

1. Elegir respaldo, revisión compatible y entorno aislado. Detener correo externo y otras salidas; proteger datos personales y no enviar invitaciones reales durante ensayo.
2. Verificar hashes, integridad de la base y disponibilidad de APP_KEY y archivos. Preparar base/almacenamiento nuevos; no sobrescribir la única copia válida.
3. Restaurar con el procedimiento del motor e imágenes del mismo corte. Configurar URL/servicios del ensayo y permisos. Evitar cookies o cachés compartidos con producción.
4. Arrancar el código compatible e inspeccionar `migrate:status`. No aplicar automáticamente migraciones posteriores antes de comprobar el respaldo base.
5. Confirmar acceso, cantidades de control, últimas operaciones esperadas, detalles de venta, pagos únicos, stock, relaciones, imágenes, asignaciones y progreso. Comparar contra el inventario del respaldo sin volcar datos personales al informe.
6. Registrar resultado y tiempos. Solo después planificar la sustitución del entorno afectado, con ventana y decisiones sobre operaciones posteriores al corte. Restaurar pierde cambios posteriores; reconciliarlos exige revisión, no duplicar cobros o stock a mano.

Si solo se cambia código, volver a una revisión compatible puede bastar. Si cambiaron esquema/datos, evaluar restauración y compatibilidad antes de rollback. No ocultar un fallo abriendo el servicio sin comprobar acceso y consistencia.

9. Mantenimiento e incidentes

Síntoma	Diagnóstico	Acción razonada
Correo no llega	Mailer log/array, transporte y remitente	Configurar transporte válido y ensayar; no repetir ilimitadamente
Google no inicia	Credenciales, callback, sesión/HTTPS	Corregir entorno; usar acceso por correo mientras tanto
419	Sesión/token caducados o cookie/dominio	Recargar formulario y comprobar sesión/HTTPS antes de repetir
403	Rol, actividad, propiedad o asignación	Revisar cuenta/vínculo; no otorgar permisos globales para eludirlo
No aparece entrenador	Rutina asignada por ID y entrenador activo	Verificar asignación autorizada; nombre histórico solo no basta
Stock insuficiente	Inventario activo y cantidades	Corregir carrito; ajuste de existencias solo Admin
Simulación no aumenta ingreso	Bandera de prueba	Comportamiento esperado; consultar historial de simulaciones
Error al cargar imagen	Tamaño, píxeles, formato y GD/WebP	Corregir archivo/requisitos; no publicar bytes originales
CSS/JS no carga	Manifest, archivos y public/hot	Publicar build correcto y comprobar respuesta HTTP
DB bloqueada o migración falla	Motor, escritores, permisos y schema	Registrar causa, detener escrituras propias y retomar paso seguro

Registrar hora, revisión, rol, operación, error acotado y modo de reproducción sin secretos. Conservar evidencia y respaldo antes de intervenir datos. Supervisar disponibilidad, errores, espacio, certificados, fallos de correo y funcionamiento de respaldos. No hay alertas externas ni vigilancia continua configuradas por este documento.

10. Pendientes del entorno

Para publicación faltan decisiones/configuración institucional: correo real con prueba de entrega, responsable/contacto de privacidad y dominio/HTTPS/servidor y respaldos del destino. Google solo requiere credenciales y prueba si se decide habilitarlo. La simulación de ventas está implementada; conectarla a pagos reales sería otro proyecto con proveedor, credenciales, estados, conciliación y pruebas autorizadas.

La entrega local permite revisar y demostrar las funciones con datos sintéticos. Los comandos descritos no publican por sí solos cambios en GitHub ni crean un alojamiento.